

Recovering drift fields from random walks

Six estimators, one set of walkers, and where each one wins

The dfi package • 46 tests • every number reproduced by a numbered script

The question. Particles move by $dx = b(x) dt + \sqrt{2D} dW$. From their observed motion, recover the vector field b . The instantaneous signal-to-noise ratio is about 10^{-3} : over one step the drift contributes $b\Delta t$ and the noise $\sqrt{2D\Delta t}$, so every method is an exercise in averaging away a thousandfold larger random term without averaging away the field.

What was built. A simulator, six estimators of increasing structure, and one measurement protocol that runs all of them on the *same* walkers, so that differences are differences between methods and not between experiments.

The three results worth the reader's time. (i) A network trained across many simulated fields beats every classical estimator by $\sim 40\%$ – and loses to the simplest kernel on fields from a generator it never saw. (ii) At equilibrium, *static positions* carry more about the drift than the increments do, decisively so in 5D. (iii) Static positions are *exactly* blind to circulation, which turns the pair of families into a measurement of entropy production.

1 The problem, and why it splits in two

The overdamped Langevin equation $dx = b(x) dt + \sqrt{2D} dW$ is the standard model for a particle in a force field with thermal agitation, and its inverse problem – recover b from observed motion – appears in single-particle tracking, active matter, ecology, and anywhere a stochastic differential equation is fitted to a trajectory. Two kinds of data support two fundamentally different answers.

Trajectories. Positions with time order, sampled at spacing Δt . The Kramers–Moyal relation makes the whole drift a conditional average,

$$b(x) = \lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \mathbb{E}[\Delta x | x], \quad \text{so} \quad y_i \equiv \frac{\Delta x_i}{\Delta t} = b(x_i) + \text{noise of size } \sqrt{2D/\Delta t}. \quad (1)$$

Every increment is an unbiased but violently noisy sample of the field at the point it started from. Estimating b is a regression of y on x in which the noise dominates the signal by three orders of magnitude, and the six estimators below differ only in how they pool neighbouring samples.

Snapshots. Positions *without* time order, drawn from the stationary density ρ_{ss} . Here the only accessible object is the score:

$$\nabla \log \rho_{ss}(x) = -\nabla U(x)/D = b_{\text{grad}}(x)/D. \quad (2)$$

The identifiability gap is not a weakness of any method. Snapshots can recover the gradient part and nothing else, whatever the estimator; trajectories can recover all of b . This project measures that boundary rather than arguing about it, and then uses it: the difference between what the two families see is the part of the dynamics that breaks detailed balance.

2 Setup and measurement protocol

Fields. Gaussian random fields on the torus $[-1, 1]^d$ with a tunable correlation length, decomposed as $b = -\nabla U + \omega A \nabla U$ with A antisymmetric, so ω dials circulation without touching ρ_{ss} . Four other generators (sparse wells, cellular patterns, mixed spectra, spatially varying anisotropy) exist for out-of-distribution tests.

Simulation. Euler–Maruyama with sub-stepping: the integrator runs at $\Delta t/\text{substeps}$ while the data are recorded at Δt . Without that separation an $O(\Delta t)$ bias in an estimator is indistinguishable from an $O(\Delta t)$

Helmholtz decomposition -- random-grf-2D

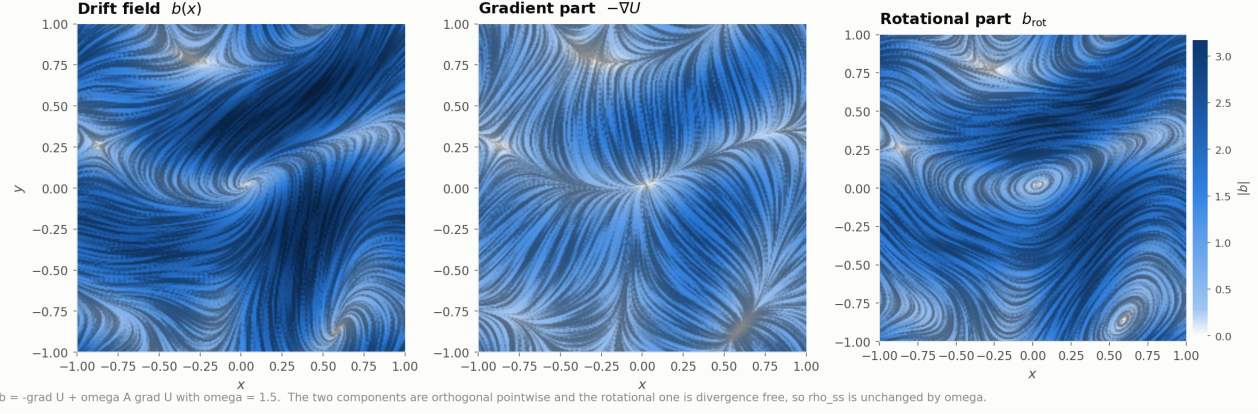


Figure 1: A drift field splits into a gradient part and a divergence-free rotational part, $b = -\nabla U + \omega A \nabla U$. The rotational part carries a steady probability current but leaves $\rho_{ss} \propto e^{-U/D}$ *exactly* unchanged — so a snapshot dataset is identical in distribution for a system with no circulation and one with a great deal of it.

error in the simulator, and one would be measuring one’s own integrator. Trajectories are cached by configuration hash, so every estimator sees bit-identical walkers.

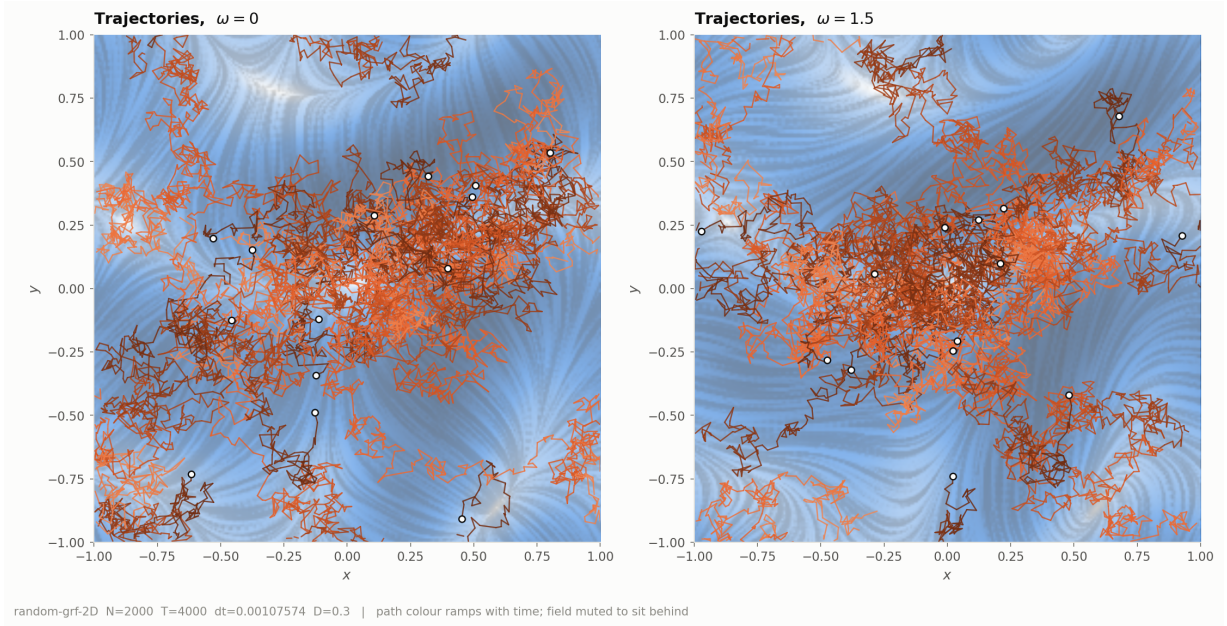


Figure 2: What the estimators are given: 2000 walkers $\times$ 4000 steps in the same field, without and with circulation ($\omega = 0$ and 1.5), paths coloured by time. The drift is invisible in any single step — it contributes $b\Delta t \approx 10^{-3}$ of displacement against a random $\sqrt{2D\Delta t} \approx 2.5 \times 10^{-2}$ — and appears only after averaging thousands of them. The circulation on the right is likewise not visible by eye in the paths, but it is recoverable from their increments and provably absent from their positions.

Score. Normalised RMS error against the true field on a dense grid, $\text{nmse} = \|\hat{b} - b\|/\|b\|$, so 1.0 means “as bad as predicting zero”. Reported alongside: coverage (what fraction of the box a method will answer for), the cosine of the angle, the scale ratio, and the error split into gradient and rotational components. Every method is scored on *all* evaluation points, including ones it declines to answer for — scoring only where a method claims support rewards abstention.

Tuning. Every method chooses its own smoothing by cross-validation grouped *by walker*, never by sample. Splitting increments at random lets a held-out point sit one time step away from its training neighbours, and the criterion then rewards a bandwidth far too small. This bit one estimator in the study (§4.7).

3 The ladder

	method	consumes	what it assumes
1	binned Kramers–Moyal	increments	b is constant inside a cell
2	kernel regression (NW, local-linear)	increments	b is smooth on one length scale
3	basis projection (SFI)	increments	b lies in a Fourier basis
4	neural network (MLP, CNN)	increments	smoothness implied by architecture + early stopping
4b	amortised CNN	increments, binned to a grid	what fields from this generator look like equilibrium (gradient drift)
5	KDE score, denoising score matching	positions only, plus D	

Rungs 1–3: hand-built smoothing. Binning averages increments per cell and returns one value for the whole cell; the kernel replaces the hard cell with a weighted neighbourhood, and its local-linear version removes the first-order bias that binning carries; SFI (stochastic force inference) projects the increments onto a global Fourier basis, which is exactly the right basis for these fields. Each rung has one smoothing parameter, chosen by cross-validation.

Rung 4: learned smoothing, one dataset. An MLP with periodic features, or a CNN decoding a learned latent to an M^d grid, fitted by least squares against $\Delta x / \Delta t$ on one dataset. The network never sees b ; the smoothing dial is the number of training steps, chosen on held-out walkers. Because the noise is 10^3 times the signal, the training loss is uninformative and the only readable curve is the *gain over predicting zero*, $|y|^2 - |y - \hat{b}|^2$, which on these data rises from 0 to about 10^{-3} of the total target power.

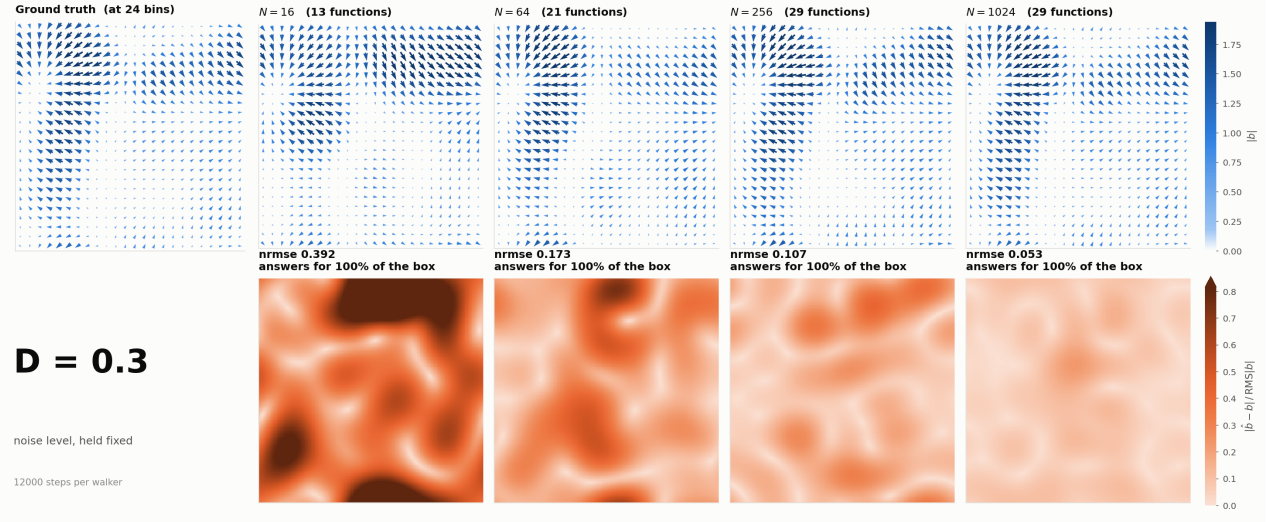
Rung 4b: amortised inference. The same architecture trained once across 3000 simulated fields, each with its own short trajectory set. At inference it reads a new dataset’s binned increments and outputs a field with no retraining and no cross-validation. The input is three physical channels — a z-scored mean increment $Y/(\sigma\sqrt{N})$, a shrunk mean $Y/(N + \lambda)$ and the shrinkage weight $N/(N + \lambda)$ with $\lambda = \sigma^2/\tau^2$ — so the network is handed the statistics rather than the raw counts. A periodic U-Net with no normalisation layers (which would destroy the physical amplitude) and D_4 dihedral test-time augmentation with proper vector-channel transforms.

Rung 5: score from static positions. Estimating $\nabla \log \rho_{ss}$ from samples is exactly what training a diffusion model does, so denoising score matching here is not an analogy: $\mathbb{E} \|\sigma s(x + \sigma \varepsilon, \sigma) + \varepsilon\|^2$ estimates the physical quantity directly. A Gaussian KDE differentiated in closed form is the classical counterpart. Both select their smoothing by held-out Hyvärinen score matching, $J(s) = \mathbb{E}[\frac{1}{2}|s|^2 + \nabla \cdot s]$, which ranks candidates by score error without knowing the truth.

4 Results

All numbers below are on the same 2D random field and the same walkers unless stated. Snapshot rows are a different kind of data on the same systems: independent positions matched in count (capped at 500k), plus the known D , and no time ordering.

	4.8M transitions	16 walkers	1024 walkers	$D = 1.0$	$\sigma_{\text{obs}} = 0.02$	$d = 3$	$d = 5$
binned KM	0.293	0.648	0.224	0.785	1.56	0.564	1.62
local-linear kernel	0.126	0.358	0.088	0.280	1.25	0.261	0.925
SFI (basis)	0.097	0.392	0.053	0.211	1.43	0.195	0.946
MLP	0.100	0.340	0.080	0.319	1.38	0.223	0.927
CNN (per dataset)	0.115	0.380	0.068	0.333	1.44	0.224	—
amortised CNN	0.060	0.257	0.043	0.134	0.63	—	—
KDE score (snapshots)	0.130	0.132	0.130	0.843	0.14	0.231	0.983
DSM (snapshots)	0.072	0.194	0.072	0.164	0.08	0.142	0.625



Basis projection (SFI). The basis size is chosen by cross-validation over walkers, so the estimate keeps fewer Fourier modes as the data thins – smoother, missing fine structure rather than noisier. That is the bias-variance trade being made for you; figure 04 is the same trade made by hand.

Figure 3: Rung 3 as the data grows: ground truth, then estimates from 16 to 1024 walkers, with the error map below each. The basis size is chosen by cross-validation over walkers, so the estimate keeps fewer modes as the data thins – it goes smooth and misses fine structure rather than going noisy. Every rung produces this same figure on the same walkers.

4.1 More smoothing structure beats more flexibility – until the structure is wrong

Ranked at 4.8M transitions the order is binning (0.293), kernel (0.126), network (0.100), basis (0.097). Each step buys accuracy by assuming more, and the assumptions happen to be true: the fields really are sums of Fourier modes, so SFI’s global basis is not a convenience but a match. The advantage disappears the moment the match does – on fields built from other generators (sparse wells, cellular patterns) SFI drops behind the local kernel, which assumes only smoothness.

The per-dataset network lands in the middle almost everywhere and wins where data is scarcest (0.340 at 16 walkers). Its one structural weakness is that its smoothing dial is noisy: with plenty of data the held-out curve is flat, and stopping on it costs about 5% of the error (0.082 against 0.076 at the truth-optimal step).

4.2 Amortisation is the largest single win, and it is prior knowledge

Trained once across 3000 fields, the same convolutional architecture that *loses* to an MLP on one dataset wins nearly every comparison: 0.060 against SFI’s 0.097 at the largest budget, 0.257 against 0.340 at the smallest, and 0.134 against 0.211 at high diffusion. It needs no cross-validation and no retraining at inference.

What it has bought is a prior over fields, and the price is measurable. Its equivalent smoothing kernel – the linear response $\partial \hat{b}(x_0)/\partial Y_c$ of the output to each input cell – keeps a *fixed width* as data grows and instead raises its gain from 0.60 to 0.93. That is the signature of a Wiener filter matched to the field statistics, not of a bandwidth being tuned. On fields from five other generators it loses, by up to $1.9\times$.

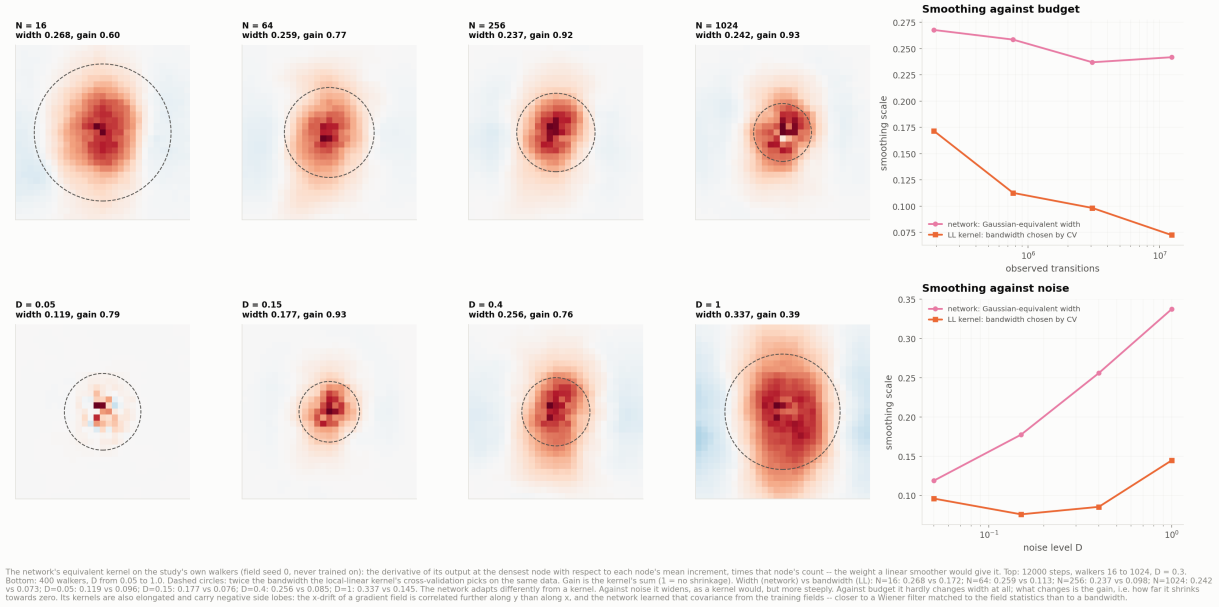


Figure 4: The amortised network's *equivalent kernel*: the linear response of one output cell to each input cell, recovered by differentiating the trained network. As the data grows the kernel does not narrow — its width is set by the field prior — it raises its gain from 0.60 to 0.93. A cross-validated bandwidth does the opposite. This is the difference between learning a smoother and learning a prior.

This is the cleanest result in the project for a reader deciding whether to amortise: the learned estimator is better because it knows what the answer should look like, and it degrades exactly as far as that knowledge is wrong. Both halves are measured, on the same walkers, with the same score.

4.3 At equilibrium, static positions beat increments

This was the unexpected one. The Kramers–Moyal target is dominated by noise of size $\sqrt{2D/\Delta t}$; a *position* is not noisy at all. On a gradient field, denoising score matching on 500k independent snapshots reaches 0.072 where SFI on a matched number of increments gets 0.097 — and using the very same walkers' positions rather than idealised samples, it still beats every trajectory method at 16 and 64 walkers (0.302 against 0.392 and 0.358).

The gap widens with dimension. At $d = 5$ with 6.1M transitions every trajectory method sits near 0.93, i.e. barely better than predicting zero, while DSM on positions reaches **0.625**. This is not a capacity limit — the same MLP fits the exact 5D drift to 0.04 when handed noiseless targets — it is information: each increment carries noise $48\times$ the drift, and the field has ~ 1700 independent correlation volumes. With $100\times$ the data both MLP and SFI reach ~ 0.2 .

4.4 Snapshots are blind to circulation, exactly and measurably

Adding a rotational part leaves ρ_{ss} unchanged, so both snapshot estimators return *bit-identical* predictions for $\omega = 0 \dots 4$ (largest difference across the five values: exactly 0.0). Their error against the rotational component is 1.00 at every ω , and against the full drift it climbs as $\omega/\sqrt{1+\omega^2}$, while trajectory methods *improve* with ω because a stronger rotation is more signal against the same noise. The amortised CNN tracks ω up to 2 and fails at 4, outside its training range of $|\omega| \leq 1.5$.

4.5 Measurement noise separates the two families completely

Position noise σ_{obs} biases the increment target multiplicatively to $b(1 + \sigma^2/(D\Delta t))$ — the cell or neighbourhood is chosen with the noisy position, so the estimator selects on the noise it then subtracts. At $\sigma = 0.02$, one percent of the box, that factor is 2.3, and every trajectory method scores above 1.2. On snapshots the same

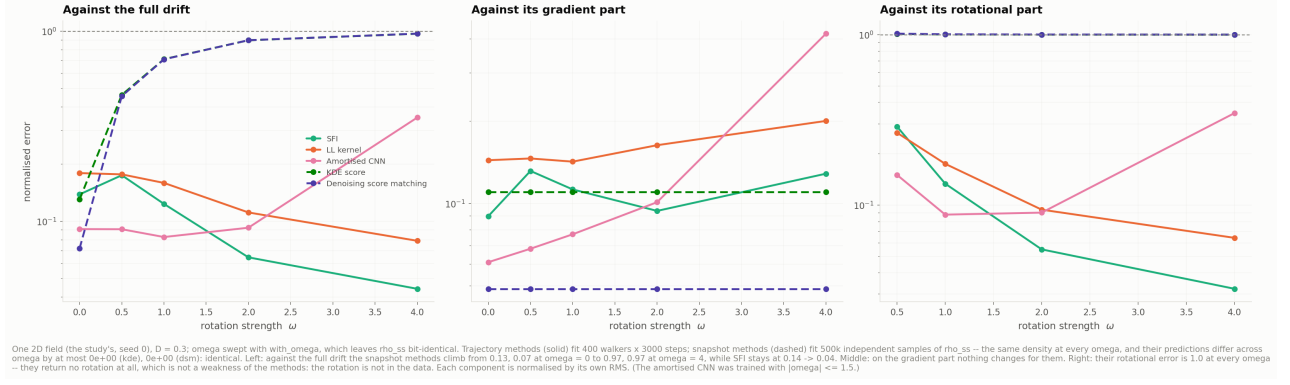


Figure 5: One field, ω swept from 0 to 4, which leaves ρ_{ss} bit-identical. Left: against the full drift the snapshot methods (dashed) climb towards 1 while trajectory methods improve. Middle: against the gradient part the snapshot methods are flat. Right: against the rotational part their error is 1.00 everywhere — they return no rotation at all.

noise merely convolves ρ_{ss} with a narrow Gaussian: the KDE score goes $0.134 \rightarrow 0.140$, DSM $0.075 \rightarrow 0.080$. The inflation formula is derived and verified term by term in the companion note `docs/binned_km_error_analysis.pdf`, where it is confirmed to 1.5% over a factor of 5 in σ , and then against this project's own 2D sweeps to within 7–10% across three values of Δt with no fitted constant.

4.6 The two families together measure dissipation

The rotational part is what the two families disagree about: $b_{\text{rot}} = b - D\nabla \log \rho$. Its magnitude is the entropy production rate $\sigma_{\text{ep}} = \langle |b - D\nabla \log \rho|^2 \rangle_{\rho} / D$, the dissipation that keeps the system out of equilibrium. Taking the drift from the increments (SFI) and the score from the positions *the same walkers* visited (DSM):

ω	0	0.5	1	2	4
exact	0	0.924	3.695	14.78	59.12
estimated	0.160	1.109	3.763	14.79	58.91
minus SFI's own predicted error	0.093	0.992	3.671	14.69	58.79

Within 2% from $\omega \geq 1$. Estimation error enters squared, so it always *adds*: at $\omega = 0$ the estimate is 0.16 instead of 0, and subtracting SFI's own noise-level error estimate — available without the truth — halves that floor. A dissipation below ~ 0.1 is not distinguishable from zero at this budget.

4.7 Every method that tunes itself can be fooled by its own criterion

Four separate instances, each caught only by comparing against the truth: cross-validation chose a 10^6 -cell grid that answered for 10% of the box (rung 1); early stopping quit 30% early on a flat held-out curve (rung 4); the KDE's bandwidth search ran to the bottom of its grid on temporally correlated positions (1.06 against 0.35 at a fixed bandwidth) because it split samples at random rather than by walker; and the same search ran to the *top* of its grid at $D = 1$, where the density is nearly flat and the criterion has almost no signal to lock onto. Every rung therefore reports what it chose, not only what it scored.

5 The error budget, in closed form

For rung 1 the whole error decomposes analytically and each term was verified against simulation. For a cell of width h holding n increments,

$$\text{MSE} \simeq \underbrace{\frac{2D}{n\Delta t}}_{\text{variance}} + \underbrace{\frac{|\nabla b|^2 h^2}{12}}_{\text{readout bias}} + \underbrace{b^2 \left(\frac{\sigma_{\text{obs}}^2}{D\Delta t} \right)^2}_{\text{noise bias}}, \quad n \simeq M\rho h^d, \quad (3)$$

where M is the total number of transitions. Substituting n gives the three facts that govern every rung:

- $\text{Var} = 2D/(M\rho h^d \Delta t) = 2D/T_{\text{occ}}$ depends only on the *time* spent in the cell. Sampling faster makes each $\Delta x/\Delta t$ noisier and gives proportionally more of them: Δt cancels.
- Balancing against the bias gives $h^* \propto M^{-1/(d+2)}$ and $\text{MSE}^* \propto M^{-2/(d+2)}$. To halve the error at $d = 6$ needs $16\times$ the data. This is the honest reason the ladder continues past rung 1.
- The measurement-noise term does *not* lose its Δt , so it diverges as $\Delta t \rightarrow 0$: with imperfect metrology, sampling faster is actively harmful.

Measured against 64 independent replicas: the variance law holds to within 1–9% at every cell width and local density, the predicted optimal bin count is 17.0 against a measured 16, and the noise inflation is confirmed to 1.5%.

6 Limitations

- **Comparisons across families are not like for like.** The amortised CNN knows the field distribution; the snapshot methods get independent samples and the true D . Both advantages are stated wherever their numbers appear, but they are advantages of the *setting*, not of the estimator.
- **One field family, one box.** Almost every number comes from Gaussian-spectrum fields on $[-1, 1]^2$. The other-generator test is the only breadth check, and it is 2D.
- D is **known** to the snapshot methods, and Δt is chosen by the simulator’s stability rule rather than by an experiment.
- **The snapshot estimators receive positions without walker labels**, so their cross-validation cannot group by walker the way the trajectory rungs do. That is a real defect in the correlated-positions setting, not only a caveat.
- **Rung 4b is 2D-only** by construction, and its 64^2 output grid caps its accuracy on nearly noise-free data.

7 What is worth doing next

1. **A hybrid estimator:** gradient part from positions, rotational remainder from increments. At 16 walkers it should beat both families; the pieces already exist.
2. **Estimate D from the data** (quadratic variation) in rung 5 and report how the drift scale degrades. It is the one input the snapshot methods still get for free.
3. **Retrain the amortised CNN on a mixture of generators** and repeat the out-of-distribution test. It should trade in-family accuracy for robustness; how much is the interesting number.
4. **A calibrated posterior:** a conditional diffusion model over drift fields giving a distribution rather than a point estimate, with coverage checked against the true field. The data generation, scoring and entropy-production checks are already here.
5. **A time-resolution sweep**, which the error budget predicts should show the $O(\Delta t)$ discretisation bias trading against the $1/\Delta t$ noise amplification.
6. **Real data** — single-particle tracking or an ecological time series — which would expose what simulation hides: unknown D , non-stationary boundaries, heterogeneous noise.

8 Engineering decisions that changed the numbers

A few implementation choices were not cosmetic, and each was forced by a measurement that came out wrong first.

An exact full-batch loss instead of minibatch SGD. For the grid-based CNN the least-squares objective over millions of increments collapses to a fixed set of lattice statistics computed once, $\sum_{c,o} G_o(c) b_c \cdot b_{c+o} - 2 \sum_c b_c \cdot Y_c + \sum_i |y_i|^2$, where G and Y are multilinear scatter sums of the data onto the lattice. The network then trains against the *exact* objective with no sampling noise, at a cost independent of the dataset size. With a target whose noise is 10^3 times the signal, the gradient noise of minibatching is not a regulariser — it is the whole signal. A test checks this loss against a float64 brute-force reference to 10^{-10} .

Sub-stepping the integrator. Data are recorded at Δt but integrated at Δt /substeps. Otherwise the $O(\Delta t)$ discretisation bias of the estimator and the $O(\Delta t)$ error of Euler–Maruyama are the same number, and the “bias” being plotted is the simulator’s.

One cache, many estimators. Trajectories are keyed by a hash of the simulation configuration, so every estimator is handed bit-identical walkers; sweep rows are cached by the same mechanism, so re-running a study redraws its figures without recomputing anything. Almost every claim in §4 is a paired comparison because of this, not an average over independent runs.

Symmetry as free data, done carefully. The amortised network is trained with D_4 dihedral augmentation, but the channels are *vectors*, and the box is a torus: a reflection must transform the vector components as well as the pixels, and on a periodic lattice it maps node $j \mapsto -j \bmod M$, i.e. a flip *and* a roll by one. Getting that wrong produces a network that is quietly worse rather than one that fails; a unit test now asserts equivariance to all eight transforms.

No normalisation layers. A U-Net with batch or group normalisation learns to produce fields of the right *shape* and the wrong *amplitude*, because the physical scale of b is exactly what normalisation removes. The scale is carried instead by the input parameterisation: shrinkage weights built from an estimate of $\sigma^2 = 2D/\Delta t$ taken from the quadratic variation of the data itself.

9 Reproducing this

command	what it produces
<code>python scripts/01_fields_and_diffusion.py</code>	the field, trajectory and snapshot figures
<code>python scripts/03_local_estimator.py</code>	the six shared figures for every estimator
<code>python scripts/04_kernel_vs_binned.py</code>	rung 2 against rung 1, bandwidth selection
<code>python scripts/05..07_amortised_*.py</code>	data, training, and the rung 4b study
<code>python scripts/08_score_study.py</code>	identifiability, entropy production, positions
<code>python scripts/09_binned_error_analysis.py</code>	the error budget of §5
<code>python -m pytest tests/</code>	46 tests, including closed-form checks

Sweep rows are cached by configuration hash in `outputs/*/data/`, so a re-run redraws the figures without recomputing anything. Full per-rung detail, with every figure and the numbers behind it, is in the `README.md` of each folder under `outputs/`; the cross-rung comparison is `outputs/SUMMARY.md`; the framework reference is `docs/dfi_framework_reference.pdf`, and the analytical note is `docs/binned_km_error_analysis.pdf`.